explain anomalies in plain language.

When a threat is detected, the system achieves sub-second security decision time with a 78% autonomous response rate — so most threats are contained without human intervention.

Translation Studio: From legacy to modern APIs

One of QBITEL Bridge's most practically useful features is the Translation Studio. Once a legacy protocol is discovered and its grammar learned, the Translation Studio can:

- Auto-generate OpenAPI 3.0 specifications from protocol grammars.
- Produce production-ready SDKs in six languages: Python, TypeScript, Go, Rust, Java, and C#.
- Create REST/gRPC API wrappers around legacy protocol sessions.

This means a COBOL mainframe system communicating over an undocumented binary protocol can be exposed as a modern REST API — without modifying the mainframe, without writing custom integration code, and with quantum-safe encryption on the wire.

Domain-specific modules

QBITEL Bridge ships with purpose-built modules for six regulated industries.

Banking and finance: ISO-8583 and SWIFT protocol handling, PCI-DSS DTMF masking for call centre voice channels, a regulatory proof engine for RBI/SEBI compliance automation, HSM integration, and multi-authority threshold signatures for cross-bank transaction authorisation.

Healthcare: EHR proxy re-encryption compatible with FHIR/HL7/DICOM, homomorphic encryption for vital sign aggregation without exposing raw patient data, ABDM-compatible verifiable credentials, and FDA 21 CFR Part 11 compliance.

Automotive: V2X (vehicle-to-everything) group signatures for multi-vehicle coordination, CAN protocol support, and misbehaviour detection for rogue vehicle identification in C-V2X networks.

Aviation: ARINC 429/629-aware aggregate signatures, ADS-B and ACARS protocol handling, and forward-secure channel establishment for flight data links.

Industrial and critical infrastructure: Modbus, DNP3, and IEC 61850 GOOSE message authentication for power grid substations, SCADA/PLC integration, and verifiable delay functions for timing-sensitive industrial control sequences.

BPO and call centres: Real-time DTMF masking, SIP/RTP quantum-safe encryption, CTI and IVR protocol support, SS7 overlay protection, and TN3270e/TN5250 mainframe session tunnelling.

Architecture and deployment

The system has four primary runtime components. Figure 3 shows how they interact.

AI engine (Python 3.11+): Comprises the ML pipeline, agent framework, LLM integration, Translation Studio, crypto layer, and REST API. It uses FastAPI, PyTorch, LangGraph for agent orchestration, ChromaDB for vector storage, and Redis for distributed caching.

Rust dataplane (Rust 1.75+): Includes high-performance packet processing, PQC-TLS termination, protocol proxying, and eBPF-based container monitoring. It's built with Tokio async runtime, and has protocol-specific adapters for ISO-8583, TN3270e, HL7 MLLP, and Modbus. Supports DPDK for hardware-accelerated packet processing.

Go controlplane (Go 1.22+): This has the management API, Kubernetes operator, Istio/Envoy xDS integration for service mesh deployments, admission webhooks, and device agent for edge deployments.

Web console (TypeScript/React): Comprises dashboard UI for monitoring, configuration, and compliance reporting.

The infrastructure is: Kubernetes-native with Helm charts, Apache Kafka for event streaming (100,000+ messages per second), Prometheus and Grafana for observability, and OpenTelemetry and Jaeger for distributed tracing.

Getting started

Run the following code:

```
# Clone the repository
git clone https://github.com/AXEwaves/qbitel-bridge.git
cd qbitel-bridge

# Install Python AI engine dependencies
pip install -r ai_engine/requirements.txt

# Build the Rust dataplane
cd rust/dataplane && cargo build --release

# Start protocol discovery on a network interface
python -m ai_engine.discovery.cli --interface eth0 --output
discovered.json

# Run with Docker Compose (includes Redis, Prometheus,
Grafana)
docker compose up -d

# Or deploy to Kubernetes with Helm
helm install qbitel-bridge ./deploy/helm/qbitel-bridge
```

Once discovery is completed, results are available via